Module 4

Continuous-Time Dynamic Programming and Reinforcement Learning

In this module, we present several essential results on continuous-time dynamic programming that are particularly important for modern research and applications of reinforcement learning (RL) to real world engineering systems. The presentation is done from the prospective of control systems engineering using feedback system decision making. The corresponding results in the discrete-time domain are presented in Module 3, where we also examine closely the relationship between the discrete-time dynamic programming and Q-learning.

Dynamic programming in continuous-time originated in the 1950s in the work of Richard Bellman, summarized in his monograph [1], and was further developed by Bellman and his coworkers [2]-[3], and many other researchers in the years to come, for example [4]-[11]. The highlight of it is the *Belman Optimality Principle* and the *Bellman functional partial differential equation* will be presented in this module.

The so-called *adaptive dynamic programming* (ADP) was derived for the first time at the beginning of this century in [12]. It is also known under the names: approximate or heuristic or neuro or iterative or incremental dynamic programming. ADP is particularly *important for the development of RL*. ADP has been considered in many journal publications, several books, and survey papers in the past twenty years, see for example [13]-[19] and references therein.

In this module, we first present in Section 4.1, the Bellman functional partial differential equation. In Section 4.2, we specialize the results obtained to the case of linear dynamic systems with quadratic performance criteria (LQ-problem), the case that can be completely solved analytically, which appears in many applications, and the case that is very important for RL. As a matter of fact, the algorithm of [8] represents an RL policy iteration algorithm for solving the optimal LQ problem. In Section 4.3, affine nonlinear optimal systems are considered providing foundation for iterative feedback policy iterations to be presented in Module 5 [9]-[10]. In Section 4.4, a RL design project is formulated for the LQ optimal feedback control problem.

4.1. The Bellman Functional Partial Differential Equation

Consider a general time-invariant, continuous-time, nonlinear dynamic system represented by

$$\frac{dx(t)}{dt} = f(x(t), u(t)), \qquad x(t_0) = x_0 \tag{4.1}$$

where $f(x(t),u(t)) \in \mathrm{R}^n$ is a continuously differentiable vector function of its arguments, $x(t) \in \mathrm{R}^n$ is the system state space vector, $u(t) \in \mathrm{R}^m$ is the system control input vector (action vector), $x(t_0) = x_0$ is the system initial state, and t stands for continuous time with t_0 representing the initial time. A scalar performance criterion, to be optimized (minimized), associated with this dynamic system, is given by

$$J\left(x(t_0),t_0\right) = \int_{t_0}^{t_f} g(x(\tau),u(\tau))d\tau + \gamma(x(t_f)) \tag{4.2}$$

where $g(x(t),u(t))$ is a *bounded nonnegative scalar function* continuously differentiable of its arguments, representing the instant value of the performance criterion (cost, award) at the continuous time instant t, t_f *is the final time assumed to be fixed*, and $\gamma(x(t_f)) \geq 0$ is the final time constraint imposed on the state variables. Note that in many applications the terminal constraint is not imposed, in which case $\gamma(x(t_f)) = 0$. The solution to the dynamic optimization problem defined in (4.1) and (4.2) is based on the *Bellman Optimality Principle*, which can be stated as follows: [5] "An optimal policy has a property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision".

The goal is to find the *optimal feedback control* input $u^{\mathrm{opt}}(x(t_f))$ (optimal policy as a function of state variables) that minimizes the performance criterion defined in (4.2). In general, the *optimal performance value* for the system that starts, in general, at some *initial state* $x(t)$, with the *initial time* t and fixed terminal time is defined by

$$J^{\mathrm{opt}}\left(x(t),t\right) = \min_{u(x(\tau)),t\leq\tau\leq t_f} \left\{ \int_{t}^{t_f} g(x(\tau),u(\tau))d\tau + \gamma(x(t_f)) \right\} \tag{4.3}$$

The *Bellman functional partial differential equation in continuous-time*, whose solution determines the optimal performance criterion value at any initial time, can be derived from (4.1) and (4.3) using the optimality principle. It is given by

$$-\frac{\partial J^{\mathrm{opt}}(x(t),t)}{\partial t} = \min_{u(x(t))} \left\{ g(x(t),u(t),t) + \left(\frac{\partial J^{\mathrm{opt}}(x(t),t)}{\partial x} \right)^{\mathrm{T}} f(x(t),u(t)) \right\} \tag{4.4}$$

This Bellman functional equation can be represented in terms of a scalar function known as the Hamiltonian. The Hamiltonian of the considered dynamic optimization problem is defined by

$$H\left(x(t), u(t), \frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right) = g(x(t),u(t)) + \left(\frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right)^{\text{T}} f(x(t),u(t)) \tag{4.5}$$

Using the Hamiltonian (4.5), the Bellman functional partial differential equation (4.4) can be represented as

$$-\frac{\partial J^{\text{opt}}(x(t),t)}{\partial t} = \min_{u(x(t))} H\left(x(t), u(t), \frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right) \tag{4.6}$$

For time invariant systems and when the performance criterion is independent on time (steady state optimization, infinite horizon optimization $t_f \to \infty$), the partial time derivative is equal to zero, in which case the Bellman functional partial differential equation becomes

$$0 = \min_{u(x(t))} H\left(x(t), u(t), \frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right) \tag{4.7}$$

Assume that the optimal control can be uniquely determined from (4.7) by using the necessary condition for minimum, and solving the corresponding algebraic equation, that is

$$\frac{\partial}{\partial u}\left\{ H\left(x(t), u(t), \frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right) \right\} = 0 \quad \Rightarrow \quad u^{\text{opt}}\left(x^{\text{opt}}(t), \frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right) \tag{4.8}$$

Having obtained the expression for optimal control in (4.8) in terms of the state variables (feedback solution), the Bellman functional partial differential equation that needs to be solved to find the optimal value of the performance criterion becomes

$$0 = H\left(x^{\text{opt}}(t), u^{\text{opt}}(x(t)), \frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right) = H\left(x^{\text{opt}}(t), \frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right) \tag{4.9}$$

with the optimal trajectory $x^{\text{opt}}(t)$ evaluated from the system differential equation defined in (4.1) using the optimal control obtained in (4.8)

$$\frac{dx^{\text{opt}}(t)}{dt} = f(x^{\text{opt}}(t), u^{\text{opt}}(t)), \qquad x^{\text{opt}}(t_0) = x_0 \tag{4.10}$$

A partial differential equation of the same form as (4.6) was derived in classical mechanics (*Hamiltonian mechanics*) in the nineteen century, where it was known under the name Hamilton-Jacobi partial differential equation. For that reason, the partial differential equation derived in (4.9) is known also in the literature under the name *Hamilton-Jacobi-Bellman (HJB) equation*.

4.2 Calculus of Variations and Continuous-Time Hamiltonian

For the continuous time-invariant, system (4.1) and the performance criterion (4.2), the dynamic optimization theory based on calculus of variations, defines the *continuous-time Hamiltonian* as in (4.5), with $p(t) = \partial J(x(t),t) / \partial x$ named the co-state variable, that is

$$H\left(x(t), u(t), \frac{\partial J^{\text{opt}}(x(t),t)}{\partial x} \right) = g(x(t), u(t)) + p^{\text{T}}(t) f(x(t), u(t)) \tag{4.11}$$

and provides the following *necessary conditions* for optimality

$$\begin{aligned}
&\frac{dx^{\text{opt}}(t)}{dt} = \frac{\partial}{\partial p}\left\{ H(x^{\text{opt}}(t), u^{\text{opt}}(t), p^{\text{opt}}(t)) \right\}, \quad x^{\text{opt}}(t_0) = x_0 \\
&\frac{dp^{\text{opt}}(t)}{dt} = -\frac{\partial}{\partial x}\left\{ H(x^{\text{opt}}(t), u^{\text{opt}}(t), p^{\text{opt}}(t)) \right\}, \; p^{\text{opt}}(t_f) = \left.\frac{\partial J(x(t),t)}{\partial x}\right|_{t=t_f} = \frac{\partial}{\partial x}\left\{ \gamma\left(x(t_f) \right) \right\} \\
&0 = \frac{\partial}{\partial u}\left\{ H\left(x^{\text{opt}}(t), u^{\text{opt}}(t), p^{\text{opt}}(t) \right) \right\} \quad \Rightarrow \quad u^{\text{opt}}(x^{\text{opt}}(t), p^{\text{opt}}(t))
\end{aligned} \tag{4.12}$$

Assuming that the unique optimal solution that minimizes the Hamiltonian in (4.12) exists, then plugging such a solution $u^{\text{opt}}(x^{\text{opt}}(t), p^{\text{opt}}(t))$ into the state-costate equations of (4.12), a two-point boundary value problem has to be solved in order to find $x^{\text{opt}}(t)$ and $p^{\text{opt}}(t)$, that is

$$\begin{aligned}
&\frac{dx^{\text{opt}}(t)}{dt} = f\left(x^{\text{opt}}(t), u^{\text{opt}}\left(x^{\text{opt}}(t), p^{\text{opt}}(t) \right) \right), \qquad x^{\text{opt}}(t_0) = x_0 \\
&\frac{dp^{\text{opt}}(t)}{dt} = -\frac{\partial}{\partial x} g\left(x^{\text{opt}}(t), u^{\text{opt}}\left(x^{\text{opt}}(t), p^{\text{opt}}(t) \right) \right) \\
&\quad -\frac{\partial}{\partial x} p^{\text{opt}^{\text{T}}}(t) f\left(x^{\text{opt}}(t), u^{\text{opt}}\left(x^{\text{opt}}(t), p^{\text{opt}}(t) \right) \right), \quad p^{\text{opt}}(t_f) = \frac{\partial}{\partial x}\left\{ \gamma\left(x^{\text{opt}}(t_f) \right) \right\}
\end{aligned} \tag{4.13}$$

It is not easy to solve this two-point boundary value problem in general. Moreover, since we are interested for the feedback solution, that is $u^{\text{opt}}(x(t))$, the two-point value boundary value problem has to provide an analytical solution for the costate variable in terms of state variables as well, which is possible in some cases leading to the feedback solution (needed for RL)

$$u^{\text{opt}}(x^{\text{opt}}(t), p^{\text{opt}}(t)) = u^{\text{opt}}(x^{\text{opt}}(t), p^{\text{opt}}(x^{\text{opt}}(t))) = u^{\text{opt}}(x^{\text{opt}}(t)) \tag{4.14}$$

Only in rare cases it is possible to find the optimal feedback controller. One of such rare cases is the optimal LQ case (linear dynamic system and quadratic performance criterion), which is a very important case from the practical point of view due to its numerous applications. Another interesting case, which is analytically tractable, is the case of optimal control of a nonlinear affine in control input time-invariant system. These two cases are presented in the next examples.

It should be emphasized that in general, it is difficult to establish the *sufficient conditions* for optimal control, and that the sufficient condition for optimality can be found in the literature only for some special, but very important cases. In general, if in addition to satisfying the necessary conditions (4.14), the optimal control makes Hamiltonian's Hessian matrix positive definite

$$\frac{\partial^2}{\partial^2 u}\left\{H(x^{\text{opt}}(t), u^{\text{opt}}(t), p^{\text{opt}}(t)\right\} > 0 \tag{4.15}$$

then it constitutes the *sufficient condition* for a local minimum.

4.3 Linear-Quadratic Optimal Control Problem

Consider a linear, continuous-time, time-invariant system represented by

$$\frac{dx(t)}{dt} = Ax(t) + Bu(t), \qquad x(t_0) = x_0 \tag{4.16}$$

where $x(t) \in R^n, u(t) \in R^m$, A and B are constant matrices of appropriate dimensions. A scalar quadratic performance criterion, to be minimized, associated to this dynamic system, is given by

$$J\left(x(t_0), t_0\right) = \frac{1}{2}\int_{t_0}^{t_f}\left(x^{\text{T}}(t)R_1x(t) + u^{\text{T}}(t)R_2u(t)\right)d\tau + \frac{1}{2}x^{\text{T}}(t_f)P_fx(t_f) \tag{4.17}$$
$$R_1 = R_1^{\text{T}} \geq 0,\ P_f = P_f^{\text{T}} \geq 0,\ R_2 = R_2^{\text{T}} > 0$$

where the scaling factor of 0.5 is introduced for convenience due to the fact that the derivatives of quadratic forms will be taken in the follow-up derivations. Positive semidefinite symmetric matrices R_1 and P_f, and positive definite symmetric matrix R_2 are weighted matrices respectively for the state and control variables.

The Hamiltonian defined in the general form in (4.5), in this particular case becomes:

$$H\left(x(t),u(t),\frac{\partial J^{\text{opt}}(x(t),t)}{\partial x}\right)=\frac{1}{2}\left(x^T(t)R_1x(t)+u^T(t)R_2u(t)\right)+\left(\frac{\partial J^{\text{opt}}(x(t),t)}{\partial x}\right)^{\text{T}}(Ax(t)+Bu(t)) \tag{4.18}$$

Using the three necessary conditions for minimum given in (4.12), we have

$$\begin{aligned}
&\frac{dx(t)}{dt}=\frac{\partial}{\partial p}\left\{H(x(t),u(t),p(t))\right\}=Ax(t)+Bu(t), \qquad x(t_0)=x_0\\
&\frac{dp(t)}{dt}=-\frac{\partial}{\partial x}\left\{H(x(t),u(t),p(t))\right\}=-R_1x(t)-A^{\text{T}}p(t),\\
&\qquad\qquad p(t_f)=\frac{\partial}{\partial x}\left\{\frac{1}{2}x^{\text{T}}(t_f)P_fx(t_f)\right\}=P_fx(t_f)
\end{aligned} \tag{4.19}$$

$$\begin{aligned}
&0=\min_u\left\{H(x(t),u(t),p(t))\right\}=\frac{\partial}{\partial u}\left\{H(x(t),u(t),p(t))\right\}\\
&\quad\Rightarrow\quad R_2u(t)+B^{\text{T}}p(t)=0\quad\Rightarrow\quad u^{\text{opt}}(p(t))=-R_2^{-1}B^{\text{T}}p(t)
\end{aligned} \tag{4.20}$$

Note these necessary conditions for the minimum are also sufficient since condition (4.15) is satisfied, that is, $\partial^2H/\partial^2u=R_2>0$. Substituting the value of the optimal control obtained into the state equation, the following form of the state-costate equations is resulted in

$$\begin{aligned}
&\frac{dx(t)}{dt}=Ax(t)-BR_2^{-1}B^{\text{T}}p(t), \qquad x(t_0)=x_0\\
&\frac{dp(t)}{dt}=-R_1x(t)-A^{\text{T}}p(t), \qquad p(t_f)=\frac{\partial}{\partial x}\left\{\frac{1}{2}x^{\text{T}}(t_f)P_fx(t_f)\right\}=P_fx(t_f)
\end{aligned} \tag{4.21}$$

The solution of this state-costate two-point boundary value problem can be found by using the Riccati formalism that assumes the existence of the relationship $p(t)=P(t)x(t)$, which leads to the following differential equation for the unknown matrix $P(t)$

$$-\frac{dP(t)}{dt} = A^{\mathrm{T}}P(t) + P(t)A + R_1 - P(t)BR_2^{-1}B^{\mathrm{T}}P(t), \qquad P(t_f) = P_f \tag{4.22}$$

This is the *matrix Riccati differential equation*. Having obtained the solution for $P(t)$, the optimal feedback control is given by

$$u^{\mathrm{opt}}(x(t)) = -R_2^{-1}B^{\mathrm{T}}P(t)x^{\mathrm{opt}}(t) = -F^{\mathrm{opt}}(t)x^{\mathrm{opt}}(t) \tag{4.23}$$

where

$$\frac{dx^{\mathrm{opt}}(t)}{dt} = \left(A - BF^{\mathrm{opt}}(t)\right)x^{\mathrm{opt}}(t), \quad x^{\mathrm{opt}}(t_0) = x_0 \tag{4.24}$$

with $F^{\mathrm{opt}}(t)$ representing the *optimal time-varying feedback gain*. It should be emphasized that the obtained optimal feedback gain is time varying, even though the original system is time invariant, which makes the optimal closed loop-system (4.24) time varying.

It is important to point out that when the linear-quadratic optimization procedure is performed over an infinite time period, $t_f \to \infty$, $P_f = 0$, then under controllability and observability conditions [20] imposed respectively on the pairs (A, B) and $(A, \mathrm{Chol}(R_1))$, where $\mathrm{Chol}(R_1)$ stands for the Cholesky decomposition of the weighted matrix R_1, the differential Riccati equation becomes the matrix *algebraic Riccati equation*

$$0 = A^{\mathrm{T}}P + PA + R_1 - PBR_2^{-1}B^{\mathrm{T}}P \tag{4.25}$$

The *unique positive definite constant stabilizing solution* P exists under the above controllability-observability conditions [21]. This solution produces the constant value for the optimal feedback gain and the time-invariant optimal linear feedback system leading to

$$\begin{aligned} u^{\mathrm{opt}}(x^{\mathrm{opt}}(t)) &= -R_2^{-1}B^{\mathrm{T}}Px^{\mathrm{opt}}(t) = -F^{\mathrm{opt}}x^{\mathrm{opt}}(t) \\ \frac{dx^{\mathrm{opt}}(t)}{dt} &= \left(A - BF^{\mathrm{opt}}\right)x^{\mathrm{opt}}(t), \quad x^{\mathrm{opt}}(t_0) = x_0 \end{aligned} \tag{4.26}$$

The controllability-observability condition can be replaced by a weaker stabilizability-detectability condition [20], that guarantees the existence of the *unique positive-semidefinite stabilizing solution* of the algebraic Riccati equation [21], which makes the closed-loop (feedback) system asymptotically stable.

Under the time-invariant ($t_f \to \infty$) optimal feedback control (4.26), the optimal performance can be evaluated as follows.

$$J^{\text{opt}}\left(x(t_0)\right) = \frac{1}{2}\int_{t_0}^{\infty} x^{\text{opt}^{\text{T}}}\left(R_1 + F^{\text{opt}^{\text{T}}} R_2 F^{\text{opt}}\right) x^{\text{opt}}(t)dt$$

The value of tis integral along trajectories of the optimal feedback system (4.26) is equal to [20]

$$J^{\text{op}t}\left(x(t_0)\right) = \frac{1}{2} x^{\text{opt}^{\text{T}}}(t_0) V x^{\text{opt}}(t_0)$$

where V satisfies the following algebraic Lyapunov equation

$$\left(A - BF^{\text{op}t}\right)^{\text{T}} V + V\left(A - BF^{\text{opt}}\right) + R_1 + F^{\text{opt}^{\text{T}}} R_2 F^{\text{opt}} = 0$$

Plugging into this equation the value for the optimal gain $F^{\text{opt}} = -R_2^{-1} B^{\text{T}} P$, we found that the algebraic equation for V becomes identical to the algebraic Riccati equation (4.25), hence $V = P$, and the optimal performance value is given by

$$J^{\text{opt}}\left(x(t_0)\right) = \frac{1}{2} x^{\text{opt}^{\text{T}}}(t_0) P x^{\text{opt}}(t_0) \tag{4.27}$$

In conclusion, the solution of the algebraic Riccati equation determines the optimal feedback gain and the optimal performance value.

Since solving the HJB equation is difficult, Bellman [1] recommended the use of successive approximation techniques, based on either guessing an approximate initial value of the performance criterion (function) and performing corresponding iterations (iterations in the function space) or by guessing an approximate initial value of the optimal control (policy) and running corresponding iterations in the policy space. These techniques, respectively also known as the *value iterations* and the *policy iterations* hopefully converge, respectively, either to the optimal performance value or to the optimal control vector (optimal policy, optimal control strategy). One of these techniques (policy iterations) will be presented in detail in the next section for the case of LQ dynamic programming in continuous time. The policy iterations for the affine nonlinear systems will be considered in the next module.

4.4 LQ Optimal RL Design Project in Continuous-Time

The project topic is the RL mechanism (RL reinforcer) of the LQ optimal control in continuous time. The project design requires the use of MATLAB/Simulink including live scripts and live functions. This project is appropriate for senior undergraduate and first year graduate students.

Project Report Format: The project reports are supposed to be typed and contain standard technical paper parts: introduction, problem formulation, main results, conclusions, references, MATLAB codes. The report should include a brief description of the RL algorithm used with all important formulas, Simulink block diagrams if any, obtained simulation results, tables, and produced figures. The MATLAB program code should be placed at the end of the report in an appendix.

Project Tasks: Take any linear system of your interest whose state space form is known. Check that the assumptions for the existence of the LQ optimal control stated in Step 0 (see the next page) are satisfied. For simplicity, take the weighted matrices as identity matrices of the corresponding dimensions, that is, $R_1 = I_n$, $R_2 = I_m$. In MATLAB, the algebraic Lyapunov and Riccati equations can be solved using the MATLAB functions `lyap` and `are` or `lqr`. Make a table that shows how the approximate performance values converge per iteration to the optimal values. In general, you need to run only five to six iterations, in rare case nine or ten iterations since the algebraic Lyapunov iterations are in fact the Newton method iterations for solving the ARE, which will be chosen in the remaining part of this project assignment. It is known that the Newton method has the quadratic rate of convergence. You may also include the presentation about the Newton method into your report. Plot on one figure all approximates feedback controls obtained for $k = 1,2,3,4,5$, and on another figure plot the approximate trajectories for a selected (the one that performs worse in your opinion) state variable for $k = 1,2,3,4,5$.

Hint: See a demo MATLAB code given in the follow-up pages.

The LQ Optimal Reinforcement Learning Algorithm

Consider a linear, continuous-time, time-invariant system in its state space form given by

$$\frac{dx(t)}{dt} = Ax(t) + Bu(t), \qquad x(t) = x_0$$

where $x(t) \in R^n, u(t) \in R^m$, A and B are constant matrices of appropriate dimensions. A scalar quadratic performance criterion, to be minimized by using feedback control, $u(x(t))$, is as

$$J\left(x(t_0)\right)=\frac{1}{2}\int_0^{\infty}\left(x^T(t)R_1x(t)+u^T(t)R_2u(t)\right)dt,\qquad R_1=R_1^{\mathrm{T}}\geq 0,\quad R_2=R_2^{\mathrm{T}}>0$$

To solve this dynamic optimization problem via successive approximations (policy iterations of RL), the following steps [22]-[23] are needed:

Step 0: Check that the optimal LQ controller exists, that is, check that (A,B) is controllable (or at least detectable) and $(A,\mathrm{Chol}(R_1))$ is observable (or at least detectable).

Step 1: Initialization is required with an admissible (stabilizing) feedback control input, $u_1(x(t))=-F_1x(t)$, such that the feedback (closed-loop) system is asymptotically stable, that is

$$\frac{dx_1(t)}{dt}=\left(A-BF_1\right)x(t),\quad x_1(0)=x(0)=x_0$$
$$\mathrm{Re}\left\{\lambda_i(A-BF_1)\right\}<0,\ i=1,2,...,n$$

Step 2: Solve the algebraic Lyapunov equation

$$(A-BF_1)^TP_1+P_1(A-BF_1)+R_1+F_1^TR_2F_1=0\quad\Rightarrow\quad P_1$$

Note that the algebraic Lyapunov equation has a unique solution under assumption that the coefficient matrix is asymptotically stable [24]. Evaluate the approximate performance criterion as

$$J_1(x(0))=\frac{1}{2}x^T(0)P_1x(0)$$

and implement the following control to the system

$$u_2(x(t))=-R_2^{-1}B^TP_1x(t)=-F_2x(t)$$

Evaluate the approximate state trajectories and the approximate system performance values

$$\frac{dx_2(t)}{dt}=\left(A-BF_2\right)x(t),\qquad x_2(0)=x(0)=x_0\quad\Rightarrow\quad x_2(t)=e^{(A-BF_2)t}x(0)$$

$$J_2(x(0))=\frac{1}{2}x^T(0)P_2x(0),\qquad (A-BF_2)^TP_2+P_2(A-BF_2)+R_1+F_2^TR_2F_2=0\quad\Rightarrow\quad P_2$$

Observe that

$$J_1(x(0)) = \frac{1}{2}x^{\mathrm{T}}(0)P_2x(0) \quad \geq \quad J_2(x(0)) = \frac{1}{2}x^{\mathrm{T}}(0)P_2x(0)$$

Step k: Repeat the process *k*-times until the convergence to the optimal control, optimal system trajectory and the optimal performance is obtained, solve iteratively the algebraic Lyapunov equations

$$(A - BF_k)^T P_k + P_k(A - BF_k) + R_1 + F_k^T R_2 F_k = 0, \quad F_k = R_2^{-1}B^T P_{k-1}, \quad k = 3,4,5,...$$

Observe that

$$J_1(x(0)) \geq J_2(x(0)) \geq J_3(x(0)) \geq \cdots \geq J_k(x(0)) \geq \cdots \geq J_{opt}(x(0)) = \frac{1}{2}x^T(0)P_{opt}x(0)$$

Since the approximate performance criteria are given by quadratic forms, we also have

$$P_1 \geq P_2 \geq P_3 \geq \cdots \geq P_k \geq \cdots \geq P_{opt}$$

where $P_i \geq P_j$ means $P_i - P_j \geq 0$ (the deference is positive semidefinite. P_k matrices are obtained iteratively. They can be also obtained from the following algebraic Lyapunov equations after elimination of the approximate feedback gains F_k

$$(A - BR_2^{-1}B^T P_{k-1})^T P_k + P_k(A - BR_2^{-1}B^T P_{k-1}) + R_1 + P_{k-1}BR_2^{-1}B^T P_{k-1} = 0, \quad k = 2,3,4,...$$

Since the P_k sequence is convergent, for large values of k we have $P_{k-1} \to P_k \to P_\infty$ and

$$\begin{aligned}(A - BR_2^{-1}B^T P_\infty)^T P_\infty + P_\infty(A - BR_2^{-1}B^T P_\infty) + R_1 + P_\infty BR_2^{-1}B^T P_\infty \\ = A^T P_\infty + P_\infty A + R_1 - P_\infty BR_2^{-1}B^T P_\infty = 0\end{aligned}$$

Indicating that this sequence converges to the solution of the algebraic Riccati equation.

Final Step: Plot the approximate state trajectories and the approximate feedback control inputs

$$\frac{dx_k(t)}{dt} = (A - BF_k)x_k(t), \quad x_k(0) = x(0) = x_0 \quad \Rightarrow \quad x_k(t) = e^{(A-BF_k)t}x_0$$

$$u_k(x(t)) = -R_2^{-1}B^T P_{k-1}x_k(t) = -F_k x_k(t), \quad k = 2,3,4,...$$

and make a table regarding the approximate performance values and their convergence.

Note that it can be shown by Kleinman in [8] that the presented successive approximation algorithm of [22]-[23] has the quadratic rate of convergence (10^{-1},10^{-2},10^{-4},10^{-8},10^{-16}) so that this algorithm converges in only a few iterations, usually 4 to 5. In rare cases, one needs 9 to 10 iterations. The algorithm is often called the Kleinman algorithm.

In the following, we present a MATLAB demo code using a plane MATLAB script, for a standard linear dynamic system: simple pendulum on a cart, named `pendulum.m`.

MATLAB RL-Design-Project Demo 4.1:

```
% Optimal LQ RL Inverted Pendulum on a Cart Demo
clear all
% Data Input: State Space Matrices
A=[0 0 1 0; 0 0 0 1; 0 40.2 -14.53 0; 0 82.56 -15.77 0];
B=[0;0;27.28;26.74];
C=[1 0 0 0;0 1 0 0];
D=[0;0];
x0=[1;1;1;1];% assumed initial condition
% LQ Optimal Control
R1=[10 0 0 0; 0 1 0 0; 0 0 0 0; 0 0 0 0];
R2=20;
% Controllability and Observability Tests
CO=ctrb(A,B); rankCO=rank(CO)
% Forms the controllability matrix and finds the rank of
% the controllability matrix.
% The rank is equal to n (system order)implies
% the system is controllable.
OM=obsv(A,R1); rOM=rank(OM)
% Rank equal to n implies observability.
% Note that the same rank is obtained by using %
obsv(A,Chol(R1)).
% Direct Solution of the Optimal LQ Problem
[Fopt,P]=lqr(A,B,R1,R2);
Jopt=0.5*x0'*P*x0
% Reinforcement Learning Solution of the Optimal LQ Problem
ev=eig(A);
% Since one eigenvalue is in the right half plane, need to
% find F0 that stabilizes the system. To find such a F0,
% we can use the MATLAB function place that finds the
% feedback matrix that places the closed-loop matrix A-B*F0
% eigenvalues in the desired locations;
% We can have complex conjugate eigenvalues
lambda_desired=[-5; -10; -20+j*15; -20-j*15];
F0=place(A,B,lambda_desired);
% Check eigenvalues of (A-B*F0)
```

```
evF0=eig(A-B*F0);
% Vaisbod-Milstein-Kleinman Reinforcer
R2I=inv(R2);
Fi=F0
time=0:0.01:4;
for i=1:9
    Pi=lyap((A-B*Fi)',R1+Fi'*R2*Fi);
    Ji(i)=0.5*x0'*Pi*x0;
    Fi=R2I*B'*Pi;
    for j=1:1:401;
    x(:,j)=expm((A-B*Fi)*time(j))*x0;
    u(:,j)=-Fi*x(:,j);
    end
    % Presentation of the obtained results
    figure (1)
    plot(time,x(1,:)); hold on
    xlabel('Time [s]');ylabel('State variable x1'); grid
    figure (2)
    plot(time,x(2,:)); hold on
    xlabel('Time [s]');ylabel('State variable x2'); grid
    figure (3)
    plot(time,x(3,:)); hold on
    xlabel('Time [s]');ylabel('State variable x3'); grid
    figure (4)
    plot(time,x(4,:)); hold on
    xlabel('Time [s]');ylabel('State variable x4'); grid
    figure (5)
    plot(time,u); hold on
    xlabel('Time [s]');ylabel('Control input u'); grid
end
% Approximate optimal performance presentation
figure (6)
i=1:1:9
plot(i,Ji,'*')
grid; xlabel('Iteration number'); ylabel('Approximate optimal performance')
% It is nicer to present the changes of Ji in a table
% i=1, Ji=886.8558; i=2, Ji=520.0401; i=3, Ji=348.9477; i=4, Ji = 265.5725;
% i=5, Ji=226.3736; i=6, Ji=211.8111; i=7, Ji=209.2012; i=8, Ji = 209.1143;
% i=9, Ji=209.1142=Jopt; i=10, Ji = 209.1142
```

The results of this MATLAB script named `pendulum.m` are presented in Figure 4.1 for the optimal state variables and in Figure 4.2 for the approximate performance criterion changes until

the optimal value is achieved at nineth iteration. The obtained results are presented in Figures 4.1-4.6.

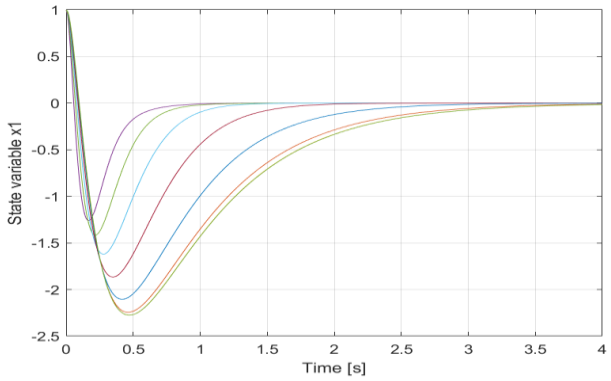


Figure 4.1: Approximate and optimal state variables $x_1(t)$

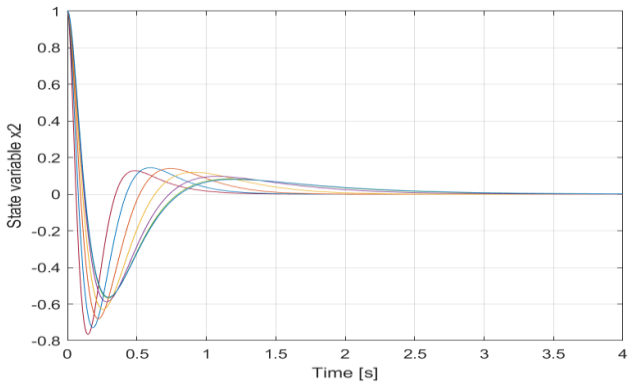


Figure 4.2: Approximate and optimal state variables $x_2(t)$

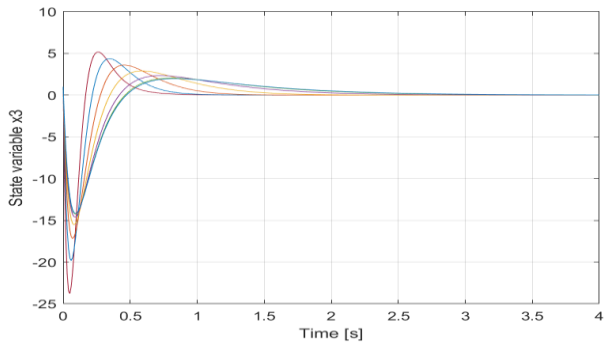


Figure 4.3: Approximate and optimal state variables $x_3(t)$

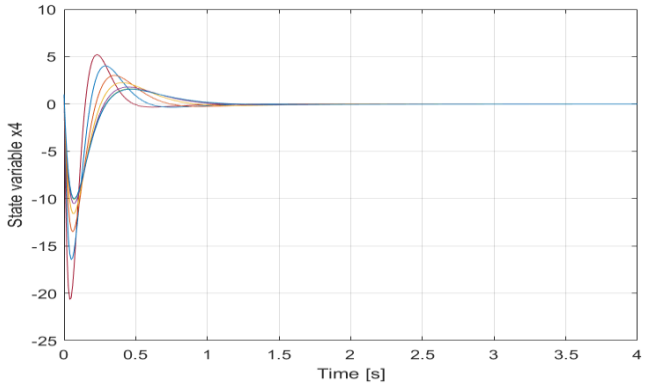


Figure 4.4: Approximate and optimal state variables $x_4(t)$

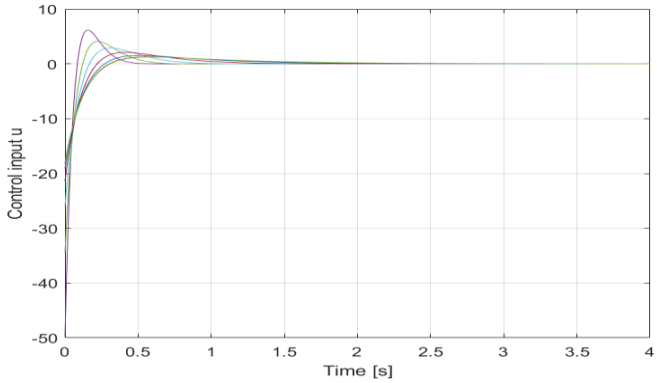


Figure 4.5: Approximate and optimal feedback controls $u(x(t))$

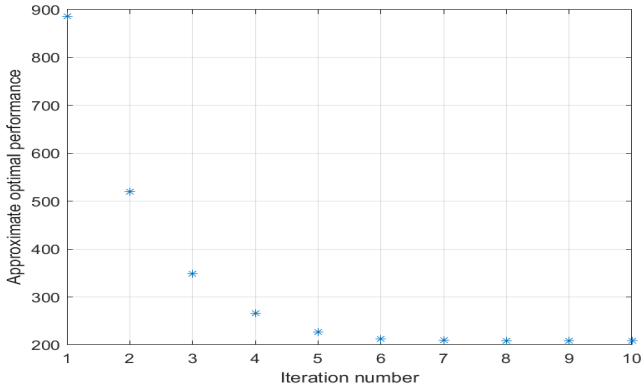


Figure 4.6: Approximate and optimal performance values per iteration

MATLAB Live Scripts and Refactoring

The students are advised to use MATLAB Live scripts in completion of their projects. As indicated in Section 2.4.1 of Module 2 to create `pendulum.mlx` from `pendulum.m` from the Editor, right-click on `pendulum.m` document tab and select "Open `pendulum.m` as Live Script". This will generate the live function `pendulum.mlx`.

The live script for the inverted pendulum on a cart demo can be further edited using the MATLAB Live Editor, for example, by replacing the percent comments with text, introducing sliders that interactively change some parameters (for example the number of iterations, the length of optimization period), and refactoring the program which will automatically replace some selected parts by live functions. *Refactoring*, we will select the input data and name the corresponding live function code `environment_data.mlx`, optimal feedback gain and optimal performance evaluation code and name it `optimal_control.mlx`, and the reinforcement learning mechanism code and name it `reinforcer.mlx`. The responses of system state trajectories are obtained from `system_responses.mlx`, and the rewards are presented by the live function `rewards.mlx`. This refactoring produces the next life script, given here with corresponding life functions:

MATLAB RL-Design-Project Demo 4.2:

Optimal LQ RL Inverted Pendulum on a Cart Demo Live Script (pendulum_continuous.mlx)

```
% Optimal LQ RL Inverted Pendulum on a Cart Demo
[A,B,x0] = environment_data;
[R1,R2,Fopt] = optimal_control(A,B,x0);
% RL Mechanism
Ji=reinforcer(A,B,R2,R1,x0);
```

```
% Presentation of the obtained results
system_response(A,B,Fopt,x0);
% Approximate optimal performance presentation
rewards(Ji);
```

% Input Data Live Function (environment_data.mlx)

```
function [A,B,x0] = environment_data
% Input Data: State Space Matrices
A=[0 0 1 0; 0 0 0 1; 0 40.2 -14.53 0; 0 82.56 -15.77 0];
B=[0;0;27.28;26.74];
C=[1 0 0 0;0 1 0 0]; D=[0;0];
x0=[1;1;1;1];% assumed initial conditions
end
```

% Optimal Control (optimal_control.mlx)

```
function [R1,R2,Fopt] = optimal_control(A,B,x0)
% LQ Optimal Control
R1=[10 0 0 0; 0 1 0 0; 0 0 0 0; 0 0 0 0];
R2=20;
% Controllability and Observability Tests
CO=ctrb(A,B); rankCO=rank(CO)
% finds the rank of the controllability matrix
% the rank is equal to n (system order)implies
% controllability
OM=obsv(A,R1); rankOM=rank(OM)
% rank equal to n implies observability
% Note that the same rank is obtained by using obsv(A,Chol(R1))
% Direct Solution of the Optimal LQ Problem
[Fopt,P]=lqr(A,B,R1,R2);
Jopt=0.5*x0'*P*x0
end
```

% Reinforcer Live Function (reinforcer.mlx)

```
function Ji = rl_environment_dynamics(A,B,R2,R1,x0)
% Reinforcement Learning Solution of the Optimal LQ Problem
ev=eig(A);
% Since one eigenvalue is in the right half plane, need to
% find F0 that stabilizes the  system. To find such a F0,
% we can  use the MATLAB function place that finds the
% feedback matrix that places the closed-loop matrix A-B*F0
% eigenvalues in the desired locations;
% can have complex conjugate eigenvalues
lambda_desired=[-5; -10; -20+j*15; -20-j*15];
F0=place(A,B,lambda_desired);
```

```
% check eigenvalues of (A-B*F0)
evF0=eig(A-B*F0);
%
% Vaisbod-Milstein-Kleinman Reinforcer
R2I=inv(R2);
for i=1:10
    Pi=lyap((A-B*F0)',R1+F0'*R2*F0);
    i
    Ji(i)=0.5*x0'*Pi*x0
    F0=R2I*B'*Pi;
end
end
```

% System Responses Live Function (system_responses.mlx)

```
function system_response(A,B,Fopt,x0)
% Plotting state responses over 5 [s]
time=0:0.01:5;
for j=1:1:501;
    x(:,j)=expm((A-B*Fopt).*time(j))*x0;
end
figure (1)
plot(time,x(1,:))
grid; xlabel('Time [s]');ylabel('State variable x1')
figure (2)
plot(time,x(2,:))
grid; xlabel('Time [s]');ylabel('State variable x2')
figure (3)
plot(time,x(3,:))
grid; xlabel('Time [s]');ylabel('State variable x3')
figure (4)
plot(time,x(4,:))
grid; xlabel('Time [s]');ylabel('State variable x4')
end
```

Rewards (rewards.mlx)

```
% function rewards(Ji)
figure (5)
i=1:1:10
plot(i,Ji,'*')
grid; xlabel('Iteration number'); ylabel('Approximate optimal performance')
% It is nicer to present the changes of Ji in a table
% i=1  Ji=886.9 % i=2  Ji=520.0 % i=3  Ji=348.9
% i=4  Ji=265.6 % i=5  Ji= 226.4 % i=6  Ji= 211.8
% i=7  Ji= 209.2 % i=8  Ji= 209.1 = Jopt
% i=9  Ji= 209.1 % i=10 Ji= 209.1
```

4.5.1 Solution with MATLAB Scripts/Live Scripts, and Interaction with Simulink

In the MATLAB programs used so far, the system response was calculated algebraically using linear algebra since we dealt with a linear system represented in its state space form. In the case of nonlinear systems, or even affine nonlinear systems such system response evaluation might not been simple, if feasible at all. It is more convenient to use a Simulink block diagram for that purpose, the block diagram that will be incorporated in the MATLAB live script iterative loop. *Moreover, this interaction can be achieved as well using a plane MATLAB script (m-file)* as presented in the following MATLAB script, which in addition has some enhanced plotting capabilities comparing to the previous MATLAB script that presents the approximate state variables after each iteration.

MATLAB RL-Design-Project Demo 4.3:

```
% Optimal LQ RL Inverted Pendulum on a Cart Demo
clear all; close all;clc
% Input Data: State Space Matrices
A=[0 0 1 0; 0 0 0 1; 0 40.2 -14.53 0; 0 82.56 -15.77 0];
B=[0;0;27.28;26.74];
C=[1 0 0 0;0 1 0 0];
D=[0;0];
x0=[1;1;1;1];% assumed initial condition
% LQ Optimal Control
R1=[10 0 0 0; 0 1 0 0; 0 0 0 0; 0 0 0 0]; R2=20;
CO=ctrb(A,B); rankCO=rank(CO); % controllability test
OM=obsv(A,R1); rankOM=rank(OM); % observability test
% Direct Solution of the Optimal LQ Problem
[Fopt,P]=lqr(A,B,R1,R2);
Jopt=0.5*x0'*P*x0
% Reinforcement Learning Solution of the Optimal LQ Problem
ev=eig(A);
lambda_desired=[-5; - 10; -20+j*15; -20-j*15];
F0=place(A,B,lambda_desired);
% Vaisbod-Milstein-Kleinman Reinforcer
R2I=inv(R2); iter = 5;
for i=1:iter
    Pi=lyap((A-B*F0)',R1+F0'*R2*F0);
     i
    Ji(i)=0.5*x0'*Pi*x0
    F0=R2I*B'*Pi;
    model=sim(pendulum)
    time_itr(1:length(model.t),i) = model.t;
```

```
    X_1(1:length(model.t),i) = model.x(:,1);
    X_2(1:length(model.t),i) = model.x(:,2);
    X_3(1:length(model.t),i) = model.x(:,3);
    X_4(1:length(model.t),i) = model.x(:,4)
end
markers={'o','+','*','.','x','s','d','^','v','>','<','p', 'h'};
lineStyles={'-','--',':','-.'};
figure (1)
 for i=1:iter
     hold on
     Iname_1=sprintf('Iteration%d',i);
plot(time_itr(:,i),X_1(:,i),'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_1)
 end
grid on
set(gca,'GridLineStyle','--')
xlabel('Time [s]');title('State variable x_1')
legend;
figure (2)
 for i=1:iter
     hold on
     Iname_2=sprintf('Iteration%d',i);
plot(time_itr(:,i),X_2(:,i),'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_2)
 end
 grid on
 set(gca,'GridLineStyle','--')
 legend; xlabel('Time [s]');title('State variable x_2')
 figure (3)
  for i=1:iter
     Iname_3=sprintf('Iteration%d',i);
plot(time_itr(:,i),X_3(:,i),'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_3)
     hold on
 end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_3')
figure (4)
 for i=1:iter
  Iname_4=sprintf('Iteration%d',i);
plot(time_itr(:,i),X_4(:,i),'Marker',markers{mod(i,iter+1)},...
'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_4)
     hold on
 end
grid on
```

```
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_4')
```

We want to emphasize that in this program, the approximate state trajectories are evaluated by Simulink using the block diagram `pendulum.mdl` and passed back to the MATLAB workspace. The pendulum Simulink model is presented in Figure 4.7. The approximate state trajectories are represented in figures 4.8-4.11 indicating convergence to the optimal trajectories.

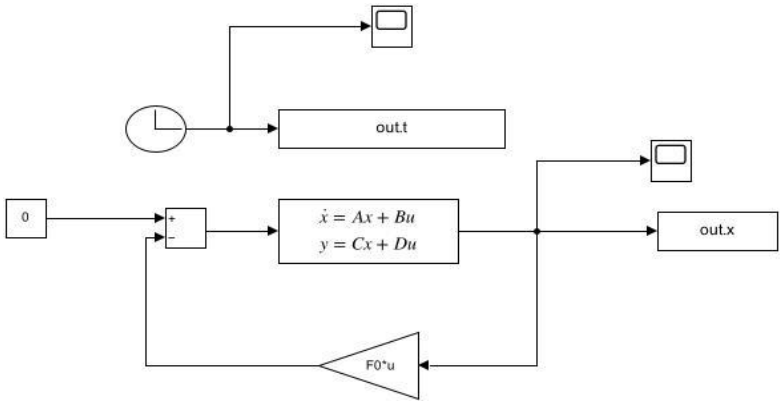


Figure 4.7: Pendulum Simulink model

The interplay of the MATLAB live script and Simulink for this example is provided in the next figure by using snipping tools in the program: MATLAB RL-Design-Project 4.4.

Several additional RL design projects for real physical linear dynamic systems in continuous-time are formulated in Section 4.6.

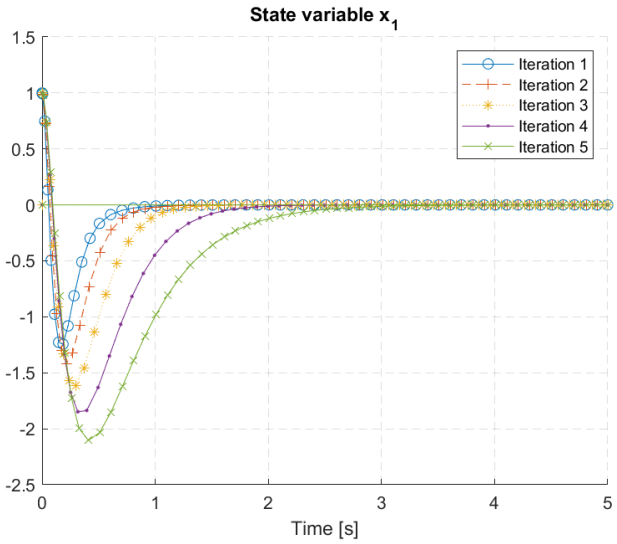


Figure 4.8: Approximate and optimal $x_1(t)$ state variable

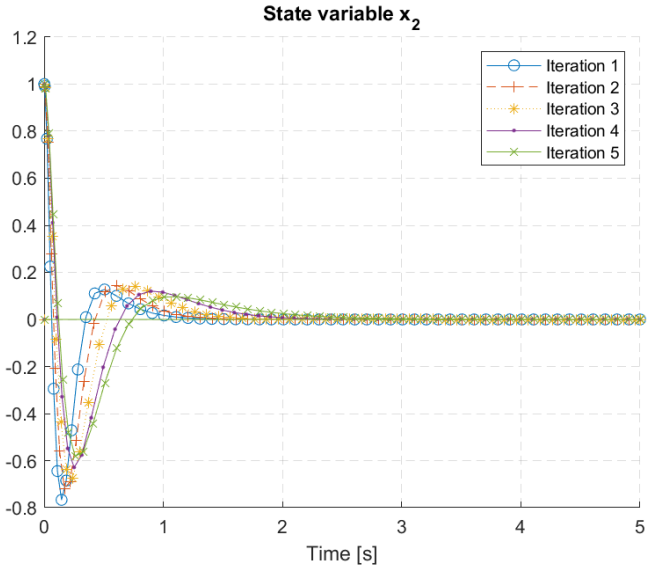


Figure 4.9: Approximate and optimal $x_2(t)$ state variable

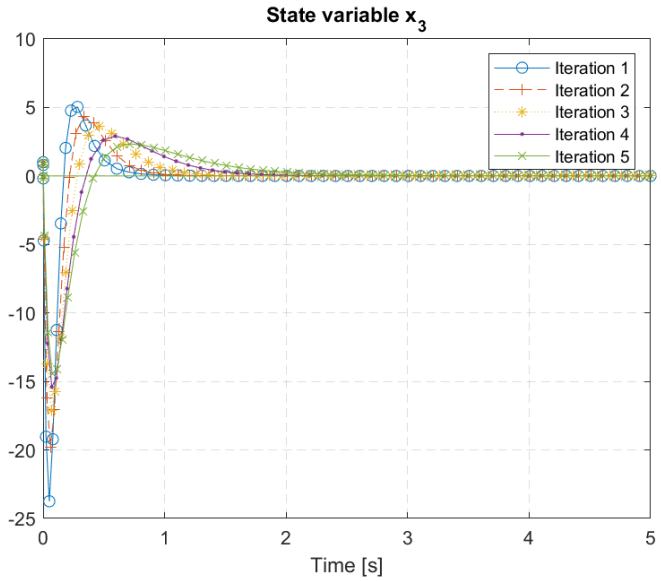


Figure 4.10: Approximate and optimal $x_3(t)$ state variable

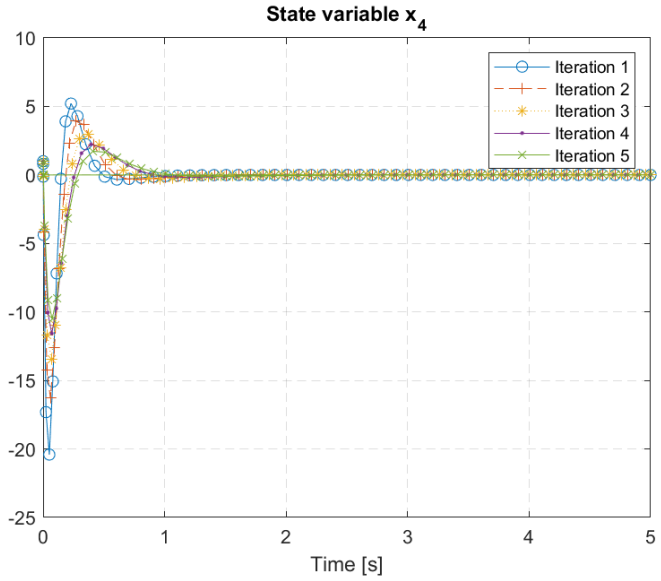


Figure 4.11: Approximate and optimal $x_4(t)$ state variable

MATLAB RL-Design-Project Demo 4.4:

Optimal LQ RL Inverted Pendulum on a Cart Demo

```
clear all; close all;
clc
```

Input Data: State Space Matrices

```
A=[0 0 1 0; 0 0 0 1; 0 40.2 -14.53 0; 0 82.56 -15.77 0];
B=[0;0;27.28;26.74];
C=[1 0 0 0;0 1 0 0];
D=[0;0];
x0=[1;1;1;1];% assumed initial condition
```

LQ Optimal Control

```
R1=[10 0 0 0; 0 1 0 0; 0 0 0 0; 0 0 0 0];
R2=20;
```

Controllability and Observability Tests

```
CO=ctrb(A,B); rankCO=rank(CO)
OM=obsv(A,R1); rankOM=rank(OM)
```

Direct Solution of the Optimal LQ Problem

```
[Fopt,P]=lqr(A,B,R1,R2);
Jopt=0.5*x0'*P*x0
```

Reinforcement Learning Solution of the Optimal LQ Problem

```
ev=eig(A);
lambda_desired=[-5; - 10; -20+j*15; -20-j*15];
F0=place(A,B,lambda_desired);
```

Vaisbod-Milstein-Kleinman Reinforcer

```
R2I=inv(R2);
iter = 5;
for i=1:iter
    Pi=lyap((A-B*F0)',R1+F0'*R2*F0);
     i
    Ji(i)=0.5*x0'*Pi*x0
    F0=R2I*B'*Pi;
    model=sim(pendulum)

    time_itr(1:length(model.t),i) = model.t;
    X_1(1:length(model.t),i) = model.x(:,1);
    X_2(1:length(model.t),i) = model.x(:,2);
    X_3(1:length(model.t),i) = model.x(:,3);
    X_4(1:length(model.t),i) = model.x(:,4);
end
markers = {'o', '+', '*', '.', 'x', 's', 'd', '^', 'v', '>', '<', 'p', 'h'};
lineStyles = {'-', '--', ':', '-.'};

figure (1)
 for i=1:iter
     hold on
     Iname_1 = sprintf('Iteration %d', i);
     plot(time_itr(:,i),X_1(:,i),'Marker',markers{mod(i,iter+1)},...
         'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_1)

 end
grid on
set(gca,'GridLineStyle','--')
xlabel('Time [s]');title('State variable x_1')
```

ErrorMessage: [0x0 char]

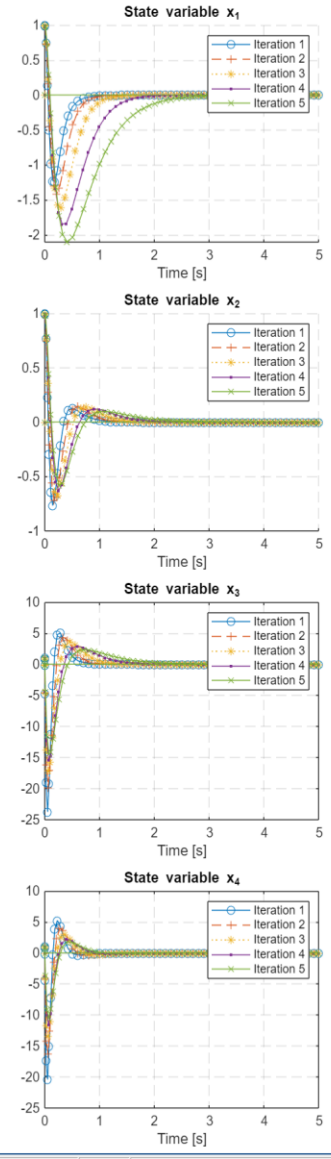


Figure 4.8: Interplay of MATLAB live scripts and Simulink for the project.

4.5.2 Communication Satellite RL Design Project Demo

The sixth-order mathematical model of a communication satellite was presented in Section 2.5 of Module 2, Problem 2.12, where the satellite angular velocity and its initial conditions were also given. The data were taken from journal paper [25]. It is interesting to observe that the same mathematical model can be used for a spacecraft orbiting the Earth.

In this design project, we used the MATLAB plane code(script) that calls Simulink that represents the satellite and provides information at all times about its state variables, and hence the approximate controls (actions) and approximate performance (cost, reward).

MATLAB RL-Design-Project Demo 4.5:

```
% Optimal LQ RL for a Satellite Demo
% Calling Simulink from MATLAB plane code loops
clear; close all; clc
omega=7.222*10^(-5);
% Input Data: State Space Matrices
A=[0 0 0 1 0 0;
   0 0 0 0 1 0;
   0 0 0 0 0 1;
   3*omega^2 0 0 0 2*omega 0
   0 0 0 -2*omega 0 0;
   0 0 (-1)*omega^2 0 0 0]
B=[zeros(3,3);eye(3)]
C=[eye(3) zeros(3,3)]
D=zeros(3,3)
x0=[10000; 10000; -10000; 2; 1; 3];% assumed initial condition in [18]
% LQ Optimal Control
R1=eye(6); R2=eye(3);
% Controllability and Observability Tests
CO=ctrb(A,B); rankCO=rank(CO)
% Finds the rank of the controllability matrix. If the rank is equal to n
% (system order)implies controllability
OM=obsv(A,R1); rankOM=rank(OM)
% Rank equal to n implies observability.
% The same rank is obtained by using obsv(A,Chol(R1))
% Direct Solution of the Optimal LQ Problem
[Fopt,P]=lqr(A,B,R1,R2)
Jopt=0.5*x0'*P*x0
% Reinforcement Learning Solution of the Optimal LQ Problem
ev=eig(A);
% Since one eigenvalue is in the right half plane, need to find F0 that
% stabilizes the  system. To find such a F0, we can  use the MATLAB function
```

```
% place that finds the feedback gain that places the eigenvalues of  A-B*F0
% in the desired locations including complex conjugate eigenvalues
lambda_desired=[-1; -3; -5; -7+i*8; -7-i*8; -10];
F0=place(A,B,lambda_desired);
% check eigenvalues of (A-B*F0)
evF0=eig(A-B*F0);
% Vaisbod-Milstein-Kleinman Reinforcer
R2I=inv(R2);
iter = 10;
for i=1:iter
    Pi=lyap((A-B*F0)',R1+F0'*R2*F0);
    i
    Ji(i)=0.5*x0'*Pi*x0
    F0=R2I*B'*Pi;
    % Calling Simulink
    model=sim(satellite)
    time_itr(1:length(model.t),i) = model.t;
    X_1(1:length(model.t),i) = model.x(:,1);
    X_2(1:length(model.t),i) = model.x(:,2);
    X_3(1:length(model.t),i) = model.x(:,3);
    X_4(1:length(model.t),i) = model.x(:,4);
    X_5(1:length(model.t),i) = model.x(:,5);
    X_6(1:length(model.t),i) = model.x(:,6);
    U_1(1:length(model.t),i) = model.u(:,1);
    U_2(1:length(model.t),i) = model.u(:,2);
    U_3(1:length(model.t),i) = model.u(:,3);
end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Plotting Approximate and Optimal State Trajectories
markers = {'o', '+', '*', '.', 'x', 's', 'd', '^', 'v', '>', '<', 'p', 'h'};
lineStyles = {'-', '--', ':', '-.'};
figure (1)
 for i=1:iter
     hold on
     Iname_1 = sprintf('Iteration %d', i);
     plot(time_itr(:,i),X_1(:,i),'Marker',markers{mod(i,iter+1)},...
         'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_1)
 end
grid on
set(gca,'GridLineStyle','--')
xlabel('Time [s]');title('State variable x_1')
legend;
figure (2)
 for i=1:iter
     hold on
     Iname_2 = sprintf('Iteration %d', i);
```

```
    plot(time_itr(:,i),X_2(:,i),'Marker',markers{mod(i,iter+1)},...
        'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_2)
 end
 grid on
 set(gca,'GridLineStyle','--')
 legend; xlabel('Time [s]');title('State variable x_2')
 figure (3)
  for i=1:iter
     Iname_3 = sprintf('Iteration %d', i);
     plot(time_itr(:,i),X_3(:,i),'Marker',markers{mod(i,iter+1)},...
         'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_3)
     hold on
 end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_3')
figure (4)
 for i=1:iter
     Iname_4 = sprintf('Iteration %d', i);
     plot(time_itr(:,i),X_4(:,i),'Marker',markers{mod(i,iter+1)},...
         'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_4)
     hold on
 end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_4')
figure (5)
 for i=1:iter
     Iname_5 = sprintf('Iteration %d', i);
     plot(time_itr(:,i),X_5(:,i),'Marker',markers{mod(i,iter+1)},...
         'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_5)
     hold on
 end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_5')
figure (6)
 for i=1:iter
     Iname_6 = sprintf('Iteration %d', i);
     plot(time_itr(:,i),X_6(:,i),'Marker',markers{mod(i,iter+1)},...
         'linestyle',lineStyles{max(mod(i,4),1)},'DisplayName',Iname_6)
     hold on
 end
```

```
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('State variable x_6')

% Plotting approximate optimal controls (actions)
figure (7)
   for i=1:iter
Iname_u1 = sprintf('Iteration %d', i);
plot(time_itr(:,i),U_1(:,i),'Marker',markers{mod(i,iter+1)},...
    'linestyle',lineStyles{max(mod(i,2),1)},'DisplayName',Iname_u1)
hold on
   end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('control u_1(t)')
figure (8)
   for i=1:iter
Iname_u2 = sprintf('Iteration %d', i);
plot(time_itr(:,i),U_2(:,i),'Marker',markers{mod(i,iter+1)},...
    'linestyle',lineStyles{max(mod(i,2),1)},'DisplayName',Iname_u2)
hold on
   end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('disturbance u_2(t)')
figure (9)
   for i=1:iter
Iname_u3 = sprintf('Iteration %d', i);
plot(time_itr(:,i),U_3(:,i),'Marker',markers{mod(i,iter+1)},...
    'linestyle',lineStyles{max(mod(i,2),1)},'DisplayName',Iname_u3)
hold on
   end
grid on
set(gca,'GridLineStyle','--')
legend; xlabel('Time [s]');title('disturbance u_3(t)')
```

Note that the satellite Simulink block diagram used is the same as one presented in Figure 4.7 with the appropriate system matrices, feedback gain, and corresponding dimensions. It has in addition a `to workspace` block placed in front of the system for the signal `out.u`.

The obtained approximate state trajectories and their convergence to the optimal ones are presented in Figures 4.9-4.14. The approximate and optimal system inputs (actions) are presented in Figures 4.15-4.17.

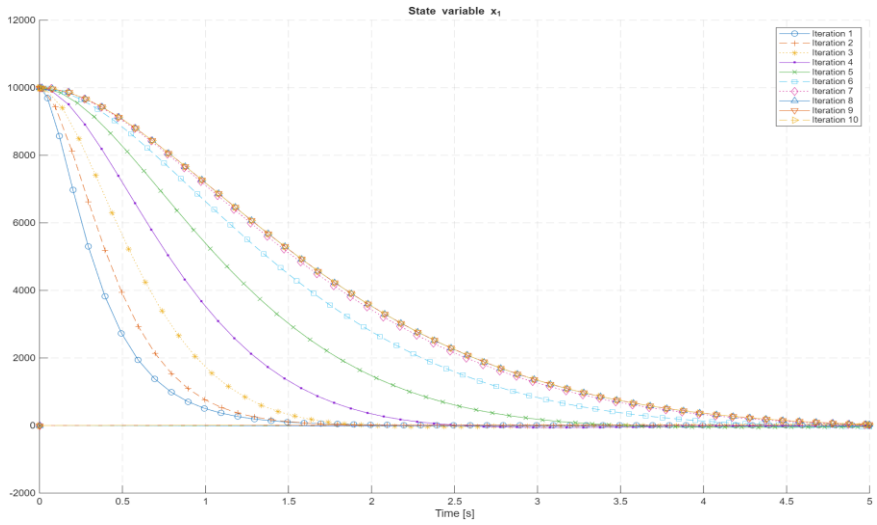


Figure 4.9: Approximate and optimal state trajectory $x_1(t)$

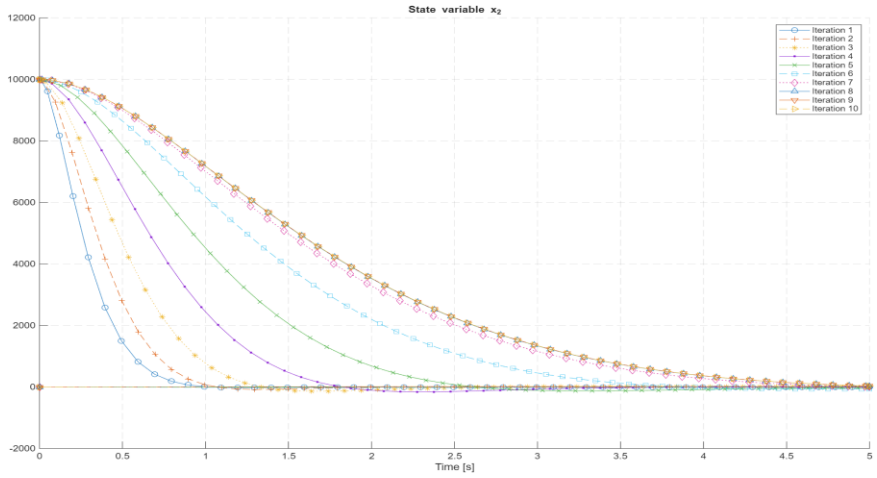


Figure 4.10: Approximate and optimal state trajectory $x_2(t)$

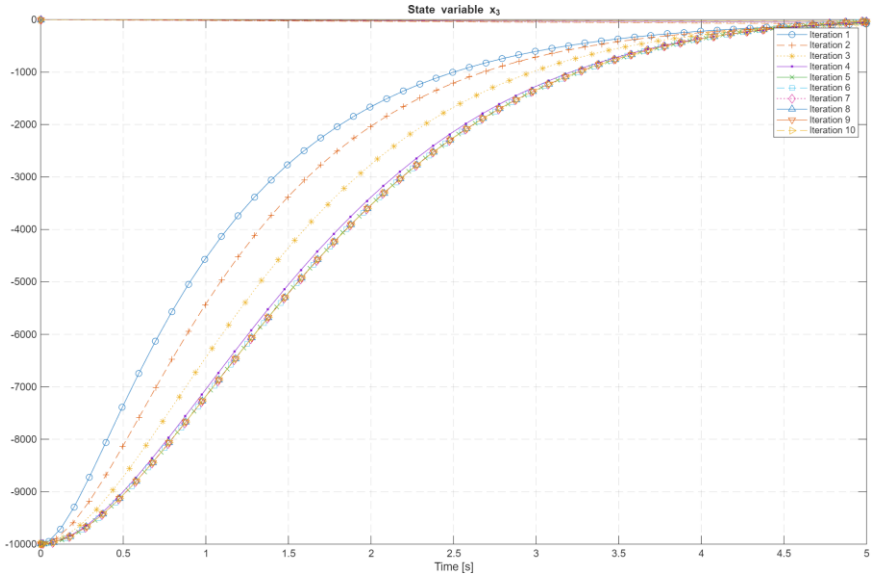


Figure 4.11: Approximate and optimal state trajectory $x_3(t)$

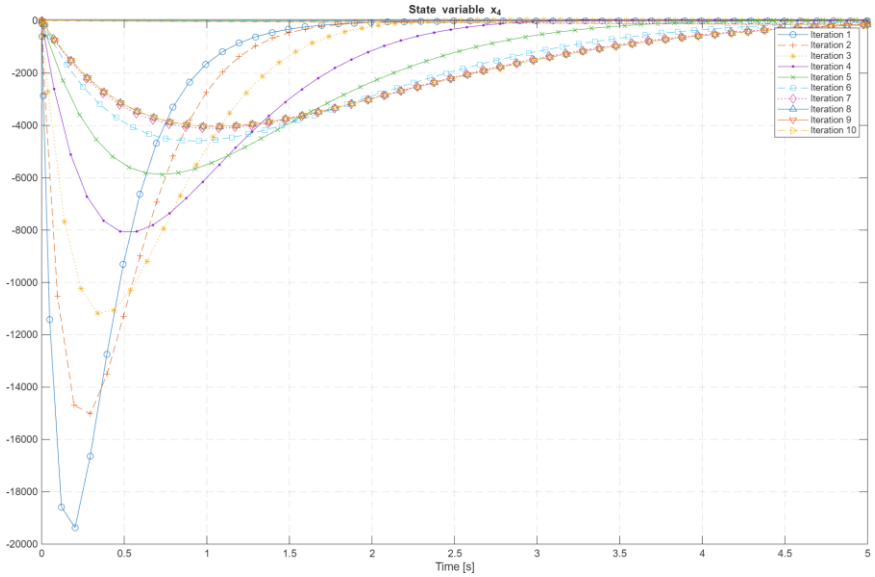


Figure 4.12: Approximate and optimal state trajectory $x_4(t)$

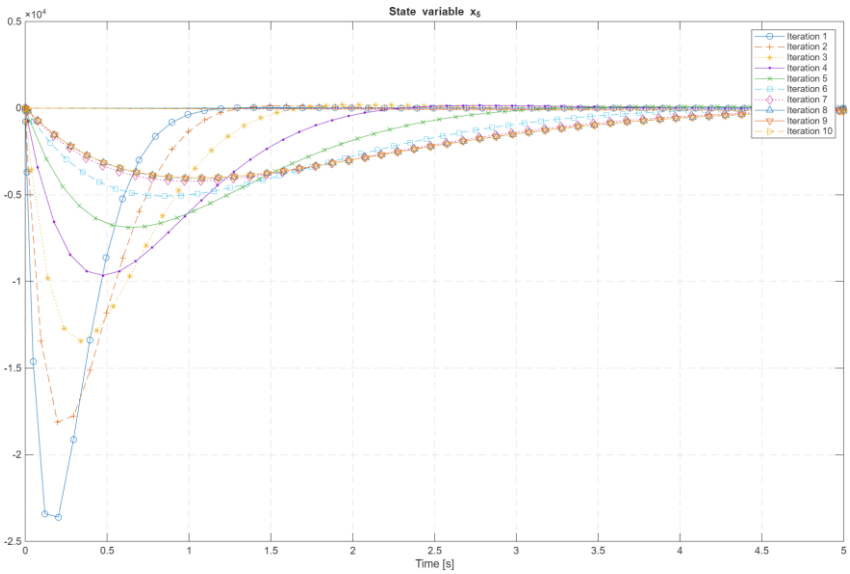


Figure 4.13: Approximate and optimal state trajectory $x_5(t)$

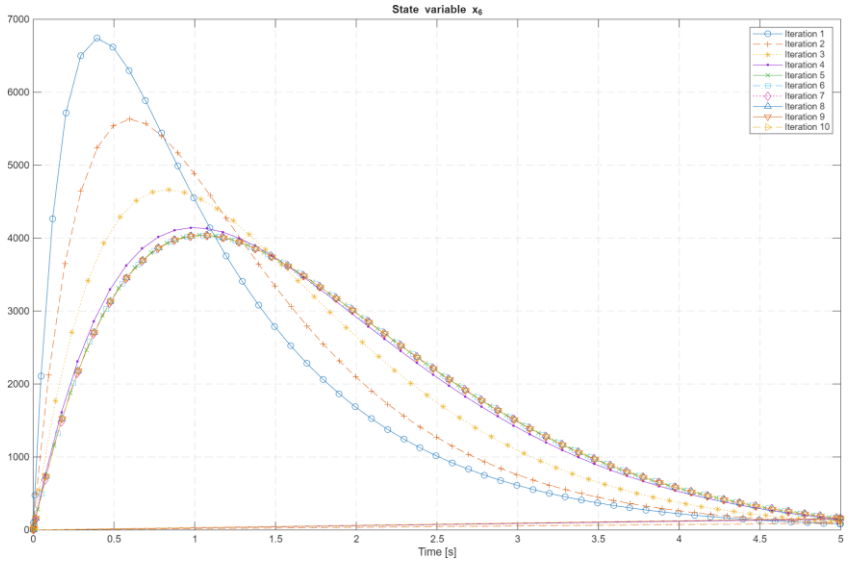


Figure 4.14: Approximate and optimal state trajectory $x_6(t)$

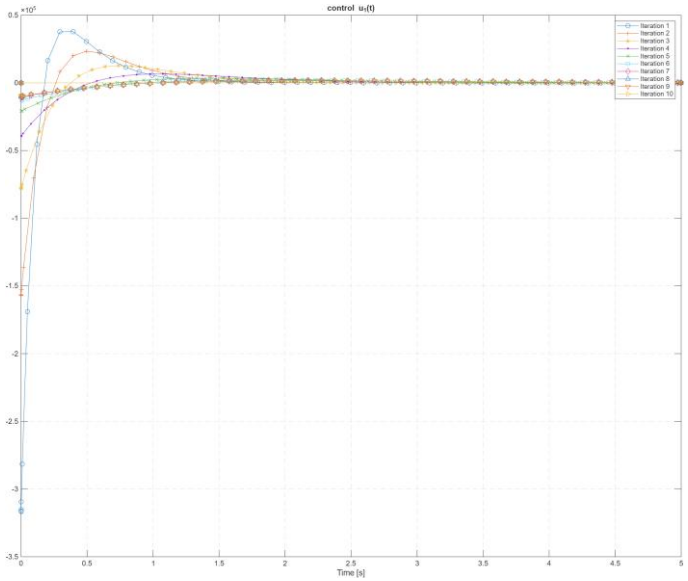


Figure 4.15: Approximate and optimal control $u_1(t)$

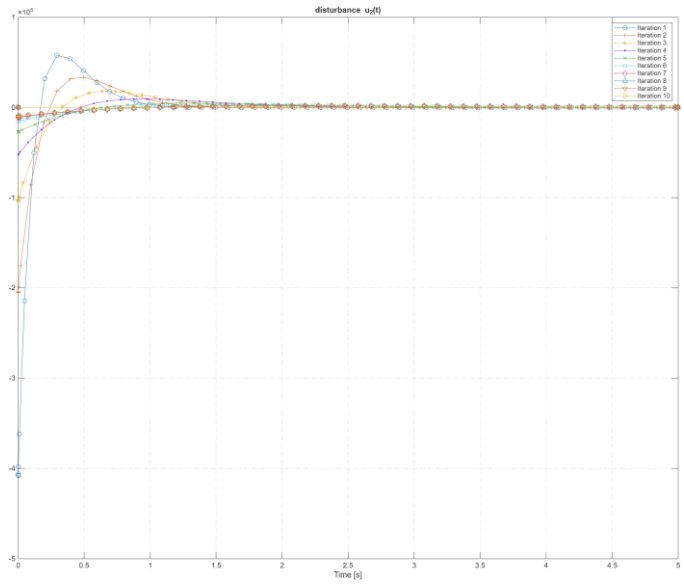


Figure 4.16: Approximate and optimal control $u_2(t)$

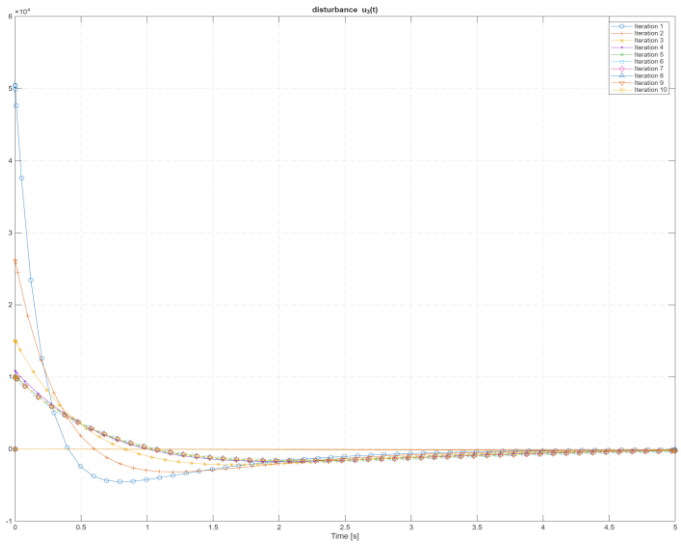


Figure 4.17: Approximate and optimal control $u_3(t)$

The approximate and optimal performance values (rewards) are presented in Table 4.1.

Iteration number	Approximate performance criterion $10^{-8} \times J_{appr}^{(i)} = 10^{-8} \times 0.5 x^T(0) P^{(i)} x(0)$
1	134.45
2	58.26
3	24.36
4	10.38
5	5.05
6	3.19
7	2.67
8	$2.60 = J_{\text{optimal}}$
9	2.60
10	2.60

Table 4.1: Approximate and optimal performance criteria for the satellit

4.6 References

[1] Bellman, R., *Dynamic Programming*, Princeton University Press, Princeton, NJ, 1957.
[2] Bellman, R. and S. Dreyfus, *Applied Dynamic Programming*, Princeton University Press, Princeton, NJ 1962.

[3] Bellman, R. and R. Kalaba, *Quasilinearization and Nonlinear Boundary-Value Problem*, Elsevier, New York, 1965.
[4] Howard, R., *Dynamic Programming and Markov Processes*, Technology Press and John Wiley and Sons, New York, 1960.
[5] Larson, R., "A survey of dynamic programming computational procedures," *IEEE Transactions on Automatic Control*, Vol. AC-xx, 767-774, 1967.
[6] Larson, R., *State Increment Dynamic Programming*, Elsevier, New York, 1968.
[7] Larson, R. and J. Casti, *Principles of Dynamic Programming: Part I*. Marcel Dekker, New York, 1978.
[8] Kleinman, D., "On an iterative technique for Riccati equations computations," *IEEE Transactions on Automatic Control*," Vol. 13, 114-115, 1968.
[9] Saridis, G. and C-S. Lee, "An approximation theory of optimal control for trainable manipulators," *IEEE Transactions on Systems, Man, and Cybernetics*, Vol. SMC-9, 152-159, 1979.
[10] Beard, R., G. Saridis, and J. Wen, 1998.
[11] Bertsekas, D. and J. Tsitsikls, *Neuro-Dynamic Programming*, Athena Scientific, Nahua, NH, 1996.
[12] Murray, J., C. Cox, G. Lendaris, and R. Saeks," Adaptive dynamic programming," *IEEE Transactions on Systems, Man, and Cybernetics- Part C: Applications and Reviews*, Vol. 32, 140-153, 2002.
[13] Vrabie, D., O.Pastravanu, M. Abu-Khalaf, and F. Lewis, "Adaptive optimal control for continuous-time linear systems based on policy iteration," *Automatica*, 477-484, 2009.
[14] Bertsekas, D., *Dynamic Programming and Optimal Control: Approximate Dynamic Programming*, Athina Scientific, Belmont, MA, 2012.
[15] Bertsekas, D., *Reinforcement Learning and Optimal Control*, Athina Scientific, Belmont, MA, 2019.
[16] Liu, D., Q. Wei, D. Wang, X. Yang, and H. Li, *Adaptive Dynamic Programming with Applications to Optimal Control*, Springer, 2017.
[17] Lewis, F. and D. Vrabie, "Reinforcement learning and feedback control," *IEEE Circuits and Systems Magazine*, 32-50, Third Quarter, 2009.
[18] Lewis, F., D. Vrabie, and K. Vamvoudakis, "Reinforcement learning and feedback control," *IEEE Control Systems Magazine*, 76-105, Dec. 2012.
[19] Wang, D., N. Gao, D. Liu, J. Li, and F. Lewis, "Recent progress in reinforcement learning and adaptive dynamic programming for advanced control applications," *IEEE/CAA Journal of Automatica Sinica*, vol. 11, 18-36, 2024.
[20] Chen, T-C., *Linear System Theory and Design*, Oxford University Press, Oxford, UK, 2012.
[21] Anderson, B. and J. Moore, *Optimal Control: Linear-Quadratic Methods*, Prentice Hall, Englewood Cliffs, NJ, 1990, republished by Dover Publications, 2007.
[22] Vaisbord, E. "An approximate method for the synthesis of optimal control," *Automation and Remote Control*, Vol. 24, 1626-1632, Dec. 1963.
[23] Milhstein, G., "Successive approximations for solution of one optimal problem," *Automation and Remote Control*, Vol. 25, 321-329, March 1964.
[24] Gajic, Z., *Lyapunov Matrix Equation in Systems Stability and Control*, Academic Press, 1995, republished by Dover Publications, 2008 (Chapter 8, Section 8.1 Kleinman's Algorithm for Riccati Equation, pages 90-95).
[25] B. Zhou, Q. Wang, Z. Lin, and G-R. Duan, "Gain scheduled control of linear systems subject to actuator saturation with application to spacecraft rendezvous," *IEEE Transactions on Control Systems Technology*, Vol. 22, 2031-2040, 2014.